Angular HandsOn 6: Services and Dependency Injection

HANDS-ON 6 [Intermediate]

Services & Dependency Injection

Topics Covered:

Creating and Providing Services[Check] providedIn: 'root' vs Component-level [Check] Providers

Injecting Services int- Components[Check] Service as a Shared Data Store[Check]

Hierarchical Dependency Injection[Check] Singleton Service Pattern[Check]

Services centralise shared logic and data that does not belong in a single component. In this hands-on, you will create Angular services for the Student Course Portal, understand the DI hierarchy, and use a service as a shared state store between components.

Task 1: Create and Use a Course Service

Goal: Build a CourseService that provides course data t- multiple components.

Steps:

58. Generate a service: ng generate service services/course. Provide it at the root level: @Injectable({ providedIn: 'root' }). In the service, define a private courses array of 5 course objects (same structure as before) and the following methods: getCourses(): Course[], getCourseById(id: number): Course | undefined, addCourse(course: Course): void.

59. Define a Course interface in a separate file (models/course.model.ts): export interface Course { id: number; name: string; code: string; credits: number; gradeStatus: 'passed' | 'failed' | 'pending'; }.

60. Inject CourseService int- CourseListComponent via the constructor: constructor(private courseService: CourseService) {}. In ngOnInit, replace the hardcoded courses array with this.courses = this.courseService.getCourses().

61. Inject the same CourseService int- HomeComponent and use getCourses().length t- display the live courses count in the stats row.

62. Add a second component (e.g., a CourseSummaryWidget) that als- injects CourseService and calls getCourses(). Verify both components share the same service instance by adding a course in one and observing the count update in the other.

Hint:

- providedIn: 'root' makes the service a singleton ? one instance shared across the entire application. All injectors up the tree share the same instance.
- Defining a TypeScript interface for your data models (Course, Student) gives you compile-time type checking across the entire application ? always prefer this over using 'any'.

Expected Outcome: Course list and home stats both read from the same service. Adding a course in one component is reflected in the other's count ? confirming the singleton instance.

Task 2: Enrollment Service and Hierarchical DI

Goal: Create an EnrollmentService and demonstrate the DI hierarchy with component-level providers.

Steps:

63. Generate an EnrollmentService: ng generate service services/enrollment. Provide it at root level. Add: private enrolledCourseIds: number[] = []; with methods enroll(courseId: number): void, unenroll(courseId: number): void, isEnrolled(courseId: number): boolean, getEnrolledCourses(): Course[] (use CourseService internally t- resolve IDs t- full Course objects).

64. Inject CourseService int- EnrollmentService ? this demonstrates service-to-service injection.

65. Inject EnrollmentService int- CourseCardComponent. When the Enroll button is clicked, call enrollmentService.enroll(course.id). Use isEnrolled(course.id) t- toggle the button label between 'Enroll' and 'Unenroll'.

66. In StudentProfileComponent, inject EnrollmentService and display the list of enrolled courses using getEnrolledCourses().

67. Demonstrate component-level providing: in a new NotificationComponent, provide a NotificationService at component level using providers: [NotificationService] in the @Component decorator. Explain in a comment why this creates a new, separate instance scoped t- that component.

Hint:

- Component-level providers create a new service instance for that component and its children ? useful when you need isolated state per component instance, such as a form wizard with multiple steps.
- Injecting one service int- another is a powerful pattern ? services can depend on other services, creating a layered architecture similar t- a backend service layer.

Expected Outcome: Enroll toggles t- Unenroll after clicking. Enrolled courses appear in the profile page. The comment explains the scoped NotificationService instance.